

Project Case Study: Autonomous Financial Research Agent

Author: Praneeth

Project Type: Agentic RAG (Retrieval-Augmented Generation) System

Development Timeline: 7 Days

Deployment Status: Live (Streamlit UI)

1. Project Rationale: Why This Project?

The financial sector is characterized by "Information Overload." Investors and analysts must jump between multiple platforms (Yahoo Finance, Google News, company websites) to gather a complete picture of an asset. This process is manual, slow, and prone to missing critical updates.

The **Autonomous Financial Research Agent** was built to solve this by creating a "Single Source of Truth." It automates the entire research workflow—from fetching live data to scraping and summarizing news—ensuring that every insight is grounded in real-time, verifiable facts.

2. The Technical Stack

We selected a "Full-Stack AI" stack optimized for speed, semantic accuracy, and zero-cost infrastructure:

Layer	Technology	Reasoning
Orchestration	LangChain	The industry standard for building modular, tool-using AI agents.
Inference	Groq (Llama 3.3 70B)	Provides ultra-fast inference speeds, making the agent's reasoning feel real-time.
Vector DB	ChromaDB	A local, persistent vector database that stores semantic embeddings without cloud costs.
Embeddings	HuggingFace	Used the <code>all-MiniLM-L6-v2</code> model for high-quality, local

		semantic processing.
Data Sources	yfinance & Serper.dev	Combined structured financial APIs with broad, unstructured web search.
Frontend	Streamlit	Allowed for the rapid deployment of a professional, interactive web interface.

3. Core Architecture: The Agentic RAG Pipeline

The project is built on the **ReAct (Reason + Act)** framework, following a logical loop:

1. **Reasoning:** The LLM analyzes the user's query and determines the best course of action.
2. **Action:** The agent selects and triggers a specialized tool (e.g., `get_stock_info` or `process_research`).
3. **Scraping & Chunking:** If researching news, the system scrapes the web, cleans the HTML with `BeautifulSoup` , and breaks text into 1000-character chunks with a 100-character overlap to maintain context.
4. **Vector Storage:** Chunks are converted into mathematical vectors and stored in **ChromaDB**.
5. **Retrieval:** The system performs a semantic search to find the Top-3 most relevant facts.
6. **Observation & Synthesis:** The LLM reads the retrieved context and generates a grounded, factual response.

4. Challenges Overcome

Building this project involved navigating significant real-world engineering hurdles:

- **Dependency & Version Management:** Encountered "Dependency Hell" with Python 3.13 and LangChain version mismatches. Solved by strategically downgrading to Python 3.11 and LangChain 0.3.x to ensure library compatibility.
- **Web Scraping Resilience:** Faced `403 Forbidden` errors from bot-protected sites (e.g., Tesla, Amazon). Implemented custom `User-Agent` spoofing and a fallback loop that iterates through multiple search results to find scrapeable content.
- **Semantic Accuracy:** Balanced the "K" value in retrieval to ensure the LLM received enough context to be accurate without exceeding token limits or introducing irrelevant

noise.

5. Skills Mastered

Through this project, I have gained proficiency in:

- **Agentic AI Design:** Implementing autonomous decision-making loops using the ReAct pattern.
 - **Advanced RAG Pipelines:** Building end-to-end retrieval systems from scraping to vector storage.
 - **Vector Databases:** Understanding semantic similarity and managing local vector stores.
 - **System Troubleshooting:** Navigating complex environment setups and library version conflicts.
 - **Full-Stack Deployment:** Bridging the gap between back-end AI logic and front-end user interfaces.
-

6. Future Roadmap

To further enhance this system, the next steps include:

- **Hybrid Search:** Combining keyword-based BM25 search with semantic search for even higher retrieval precision.
 - **Multi-Agent Workflows:** Using LangGraph to separate the "Researcher" and "Writer" into distinct, collaborating agents.
 - **Performance Evaluation:** Implementing the RAGAS framework to quantitatively measure the agent's faithfulness and relevancy.
-

This project stands as a testament to my ability to identify manual bottlenecks and engineer sophisticated, automated AI solutions that are grounded, resilient, and user-ready.